🎥 Guide Complet FFmpeg

Conversion automatisée 16:9 vers 9:16 avec arrière-plan flouté (Shorts, Reels, TikTok)

Ce guide documente la conversion automatisée de vidéos horizontales (16:9) vers un format vertical (9:16) idéal pour Shorts, Reels et TikTok, tout en conservant la vidéo originale centrée sur un arrière-plan flouté.

📌 1. Structure Globale de la Commande

Voici la commande optimisée par défaut (exécutable sous PowerShell ou Bash) :

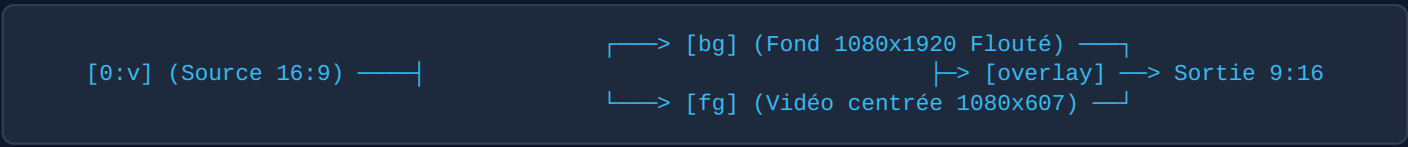
```
ffmpeg -threads 0 -i "eclair tac.mp4" -filter_complex
"[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg];
[0:v]scale=1080:-1[fg];[bg][fg]overlay=(W-w)/2:(H-h)/2" -c:v libx264 -preset superfast -crf 23 -c:a
copy "out_eclair_tac.mp4"
```

Décomposition des options de base :

- `ffmpeg` : Exécute l'outil en ligne de commande.
- `-threads 0` : Force FFmpeg à utiliser tous les cœurs du processeur disponibles.
- `-i "eclair tac.mp4"` : Fichier vidéo d'entrée (Input).
- `-filter_complex "..."` : Chaîne de filtres complexes pour le traitement vidéo multi-flux.
- `-c:v libx264 -preset superfast` : Codec vidéo H.264 avec preset ultra-rapide.
- `-crf 23` : Facteur de qualité visuelle (23 = valeur optimale qualité/taille).
- `-c:a copy` : Copie le flux audio original sans réencodage (instantané et sans perte).
- `"out_eclair_tac.mp4"` : Fichier de sortie généré.

🧩 2. Décomposition du Filtre Complexe (-filter_complex)

Le traitement s'effectue en 3 étapes séparées par des points-virgules (`;`) :



Étape 1 : Création de l'arrière-plan flou ([bg])

```
[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg]
```

- `[0:v]` : Extrait le premier flux vidéo.
- `scale=1080:1920:force_original_aspect_ratio=increase` : Agrandit la vidéo pour couvrir au minimum le cadre 1080×1920 sans la déformer.
- `crop=1080:1920` : Recadre les bords excédentaires au format 9:16 exact.
- `gblur=sigma=20` : Applique un flou gaussien optimisé.
- `[bg]` : Assigne le nom temporaire *bg* (Background).

Étape 2 : Préparation du premier plan net ([fg])

```
[0:v]scale=1080:-1[fg]
```

- `scale=1080:-1` : Redimensionne la largeur à 1080 pixels. Le `-1` indique à FFmpeg de calculer automatiquement la hauteur pour préserver le ratio d'origine (ex: 1080×607 pour une vidéo 16:9).
- `[fg]` : Assigne le nom temporaire *fg* (Foreground).

Étape 3 : Superposition et centrage

```
[bg][fg]overlay=(W-w)/2:(H-h)/2
```

- `overlay=` : Superpose le premier plan (`[fg]`) sur l'arrière-plan (`[bg]`).
- `(W-w)/2:(H-h)/2` : Formule mathématique d'alignement centré.

📐 3. Formule Mathématique et Variables

Variable	Casse	Signification	Exemple (pixels)
`W`	Majuscule	Width (Largeur de l'arrière-plan [bg])	1080 px
`H`	Majuscule	Height (Hauteur de l'arrière-plan [bg])	1920 px
`w`	Minuscule	width (Largeur de la vidéo centrée [fg])	1080 px
`h`	Minuscule	height (Hauteur de la vidéo centrée [fg])	~607 px

Calcul du Centrage :

Position Horizontale (X) :

$$X = (W - w) / 2 = (1080 - 1080) / 2 = 0 \ px$$

(La vidéo touche parfaitement les bords gauche et droit)

Position Verticale (Y) :

$$Y = (H - h) / 2 = (1920 - 607) / 2 = 1313 / 2 = 656.5 \ px$$

(La vidéo est décalée de 656.5 pixels vers le bas, la plaçant exactement au milieu de l'écran)

🎛️ 4. Contrôle de l'Intensité du Flou

Le filtre recommandé est `gblur` (Flou Gaussien), plus rapide à calculer sur processeur moderne que le traditionnel `boxblur`.

```
gblur=sigma=X
```

Valeur sigma	Effet visuel	Cas d'usage
`sigma=10`	Flou léger	Conserve des formes et éléments lisibles en arrière-plan.

Valeur sigma	Effet visuel	Cas d'usage
`sigma=20`	Flou standard (Recommandé)	Équilibre idéal entre esthétique et lisibilité de la vidéo principale.
`sigma=45`	Flou fort	Fond très diffus et abstrait.

⚡ 5. Benchmark & Comparatif de Performance

Tests réalisés sous PowerShell avec la commande `Measure-Command { ... }` sur un extrait court de 0,51 seconde (15 images).

🧪 Commandes de test (PowerShell)

1. Test CPU (libx264) :

```
Measure-Command { ffmpeg -threads 0 -i "eclair tac.mp4" -filter_complex
"[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg];
[0:v]scale=1080:-1[fg];[bg][fg]overlay=(W-w)/2:(H-h)/2" -c:v libx264 -preset superfast -crf 23 -c:a
copy "out_cpu.mp4" -y }
```

2. Test Intel QuickSync QSV (h264_qsv) :

```
Measure-Command { ffmpeg -threads 0 -i "eclair tac.mp4" -filter_complex
"[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg];
[0:v]scale=1080:-1[fg];[bg][fg]overlay=(W-w)/2:(H-h)/2" -c:v h264_qsv -preset 7 -global_quality 23
-c:a copy "out_qsv.mp4" -y }
```

3. Test Nvidia NVENC (h264_nvenc) :

```
Measure-Command { ffmpeg -threads 0 -i "eclair tac.mp4" -filter_complex
"[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg];
[0:v]scale=1080:-1[fg];[bg][fg]overlay=(W-w)/2:(H-h)/2" -c:v h264_nvenc -preset p1 -cq 23 -c:a copy
"out_nvenc.mp4" -y }
```

📊 Résultats des Tests

Encodeur / Mode	Paramètres	Temps Total	Vitesse (Speed)	Utilisation principale
CPU (libx264)	`-preset superfast`	**1,13 s**	0.563x	Processeur (Tous les cœurs)
Intel QSV (h264_qsv)	`-preset 7`	2,45 s	0.202x	Puce graphique intégrée iGPU
Intel QSV (h264_qsv)	`-preset veryfast`	2,69 s	0.183x	Puce graphique intégrée iGPU
Nvidia NVENC (h264_nvenc)	`-preset p1`	(Requiert GPU Nvidia)	N/A	Carte graphique dédiée Nvidia

🔍 Analyse Technique du Benchmark

Pourquoi le CPU est plus rapide sur les fichiers courts ?

- **Goulot d'étranglement mémoire :** Le filtre de flou `gblur` est calculé en mémoire RAM par le CPU. En utilisant `-c:v h264_qsv`, les images filtrées doivent être transférées de la RAM CPU vers la mémoire vidéo du GPU, créant un temps de transfert d'aller-retour.
- **Surcoût d'initialisation API (Overhead) :** L'initialisation des puces matérielles (Intel QSV ou Nvidia NVENC) prend quelques centaines de millisecondes, ce qui pèse lourdement sur un clip très court (0.51s).

> 💡 **Recommandations d'usage :**
>
> - **Pour des clips courts (< 1 min) ou des traitements rapides :** Utiliser le mode CPU (`libx264 -preset superfast`).
> - **Pour de longues vidéos ou du traitement par lot (Batch) :** Utiliser Intel QSV ou Nvidia NVENC pour préserver le processeur et éviter la surchauffe de l'ordinateur.

🛠️ Aide-Mémoire des Commandes Ultime

Commande Rapide CPU (Recommandée) :

```
ffmpeg -threads 0 -i input.mp4 -filter_complex
"[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg];
[0:v]scale=1080:-1[fg];[bg][fg]overlay=(W-w)/2:(H-h)/2" -c:v libx264 -preset superfast -crf 23 -c:a
copy output.mp4
```

Commande Accélération Matérielle Intel :

```
ffmpeg -threads 0 -i input.mp4 -filter_complex
"[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg];
[0:v]scale=1080:-1[fg];[bg][fg]overlay=(W-w)/2:(H-h)/2" -c:v h264_qsv -preset 7 -global_quality 23
-c:a copy output.mp4
```

Commande Accélération Matérielle Nvidia :

```
ffmpeg -threads 0 -i input.mp4 -filter_complex
"[0:v]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,gblur=sigma=20[bg];
[0:v]scale=1080:-1[fg];[bg][fg]overlay=(W-w)/2:(H-h)/2" -c:v h264_nvenc -preset p1 -cq 23 -c:a copy
output.mp4
```